Adaptive RTV Illumination Estimation for Low-Light Image Enhancement
DIP Project Proposal

Daniel Ashish Abraham 2025121010
Mohammed Adeeb 2025122005
Arun K 2025121004

1 Problem Statement

Images taken in low light are usually dark, have poor contrast and lose detail in the shadows. They also tend to be noisy, and if the brightness is simply increased, the noise increases along with it. Retinex-based methods deal with this by treating the image as the product of an illumination component and a reflectance component, $I(x, y) = R(x, y)L(x, y)$, and enhancing the reflectance (Land and McCann, 1971). How good the final image looks depends largely on how well L is estimated.

Sun et al. (2022) proposed a method that works on two copies of the input image. The first copy becomes the main feature layer and is processed with histogram equalization (HE) and Weighted Guided Image Filtering (WGIF). The second copy becomes the compensation layer. Its illumination is estimated using structure extraction from texture via relative total variation (RTV) (Xu et al., 2012), and it is then processed with MSRCR, HE, an improved bilateral gamma correction and a bilateral filter. The two layers are fused, and the fusion weights are found separately for each image using the Artificial Bee Colony (ABC) algorithm (Karaboga and Akay, 2009).

While reading the paper we noticed **two** issues.

1) The RTV step is controlled by a smoothing weight λ and a texture-suppression scale σ, which are applied as single values over the whole image. The paper does not state what values were used or how they were selected, even though this step determines the illumination estimate for the entire compensation layer.

2) The method is evaluated only with standard deviation (SD), information entropy (IE) and average gradient (AG). The authors tested 200 LOL images and report per-image values for 10 of them. These are no-reference measures, and all three tend to increase when contrast increases and also when noise is amplified, so they are not sufficient to show that one result is better than another.

2 Proposed Solution

To address these two issues, our solution has two parts. We first **reproduce the method of Sun et al. (2022) and use it as our baseline.** We then **improve the RTV illumination estimate by allowing λ to vary across the image**, which removes the dependence on a single, unexplained global value (issue 1). Because SD, IE and AG cannot show that one result is better than another, we **evaluate the change using the reference images available in the LOL dataset** (issue 2).

2.1 Reproducing the baseline pipeline

We will implement the pipeline as one program that takes a low-light image and returns the enhanced image, following the flowchart in Figure 1. The stages in the figure are parts of this program. We will check their outputs individually while debugging, but they are not separate deliverables.

Some details are missing from the paper, namely how the RTV illumination is used inside MSRCR, what the final intensity correction does, and the parameter values for RTV, WGIF and the bilateral filter. We will choose these ourselves, starting from the MSRCR settings the paper gives for its baseline comparison (scales 15, 80 and 250, $\alpha = 125$, $\beta = 46$), and record each choice in the report. For the ABC fusion step we will start with the settings given in the paper (100 iterations, a population of 45 and an abandonment limit of 90).

2.2 Adaptive RTV illumination estimation

In the second part only the RTV step is changed; the rest of the reproduced pipeline stays the same. In the RTV objective, λ multiplies the relative total

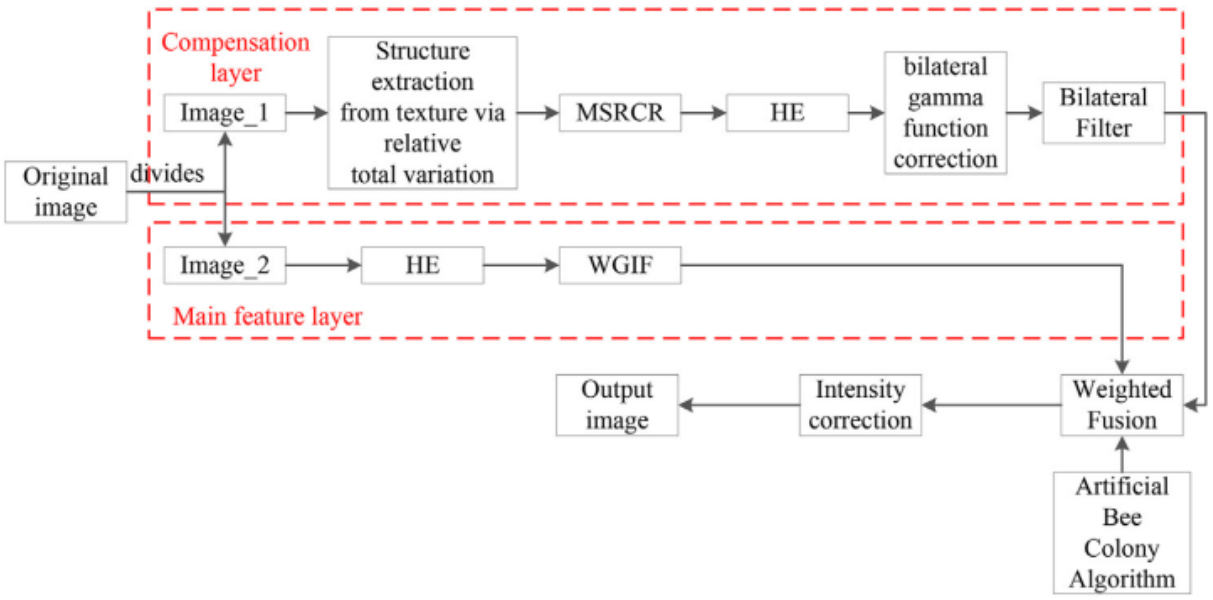


Figure 1: Overview of the reproduced enhancement pipeline.

variation term and therefore controls how smooth the illumination map becomes. We will replace the single λ with a per-pixel map $\lambda(p)$. The RTV solver already reweights the smoothness term at every iteration, so each iteration remains a sparse linear system and the same solver can be used. The variants are listed in Table 1. Variants A and B are the main work, and C and D will be tried only if time allows after A and B have been evaluated.

3 Implementation Scope

We plan to write the main algorithms ourselves in Python from scratch. This includes **RTV illumination estimation and our adaptive variants**, **MSR/MSRCR**, **HE**, **WGIF**, **bilateral gamma correction**, the **bilateral filter**, the **ABC optimizer and its fitness function**, the **fusion step**, the **LOE metric**, and the integration of all of these into one pipeline.

For lower-level operations we intend to use standard libraries: image reading and writing, colour conversion and array operations (OpenCV, NumPy), Gaussian and box filtering, the sparse linear solver used inside each RTV iteration (SciPy), PSNR and SSIM (scikit-image), and plotting (Matplotlib). If LPIPS or NIQE are added, existing implementations will be used.

4 Objectives

1. Reproduce the enhancement pipeline of Sun et al. (2022) and obtain a documented baseline on the LOL dataset.
2. Implement illumination-adaptive and texture-adaptive versions of the RTV step, and the multi-scale versions if time allows.
3. Compare these versions with the reproduced baseline using full-reference metrics, no-reference metrics and the metrics used in the paper, with significance testing.
4. Find out whether any version gives a meaningful improvement, what it costs in runtime, and on which kinds of images it helps or fails.

5 Deliverables

The main output of the project is a single pipeline in which the illumination estimator can be switched between the original RTV and our variants. The deliverables are:

1. A code repository containing the reproduced pipeline and the RTV variants.
2. Baseline results on LOL, including intermediate outputs of both layers and a list of the parameters not given in the paper together with the values we used.
3. A sensitivity analysis of λ and σ, which also provides the tuned fixed-parameter baseline.
4. A comparison of all methods: metric tables with significance tests and runtimes, and side-by-side image crops showing halos, noise or over-smoothing.
5. A final report.

Variant	Description	Priority
A. Illumination-adaptive	$\lambda(p)$ depends on the local brightness of the input. Darker regions, which have lower SNR, are smoothed more and brighter regions less.	Main
B. Texture-adaptive	$\lambda(p)$ is set from the windowed variation ratio computed inside RTV, which separates texture from structure. Textured or noisy regions are smoothed more and structural edges less.	Main
C. Multi-scale	RTV is run with several (λ, σ) pairs and the illumination maps are fused, for example weighted by local gradient confidence.	Optional
D. Combined	The better of A and B, used together with the multi-scale fusion of C.	Optional

Table 1: Proposed RTV variants.

6 Expected Outcome

We expect to have a working implementation of the method of Sun et al. (2022), measurements of how much the final image depends on the RTV illumination estimate, and a comparison showing whether a spatially varying λ performs better than a well-tuned fixed value on LOL. If it does not, we will report this and discuss the possible reasons.

References

Dervis Karaboga and Bahriye Akay. 2009. A comparative study of artificial bee colony algorithm. *Applied Mathematics and Computation*, 214(1):108–132.

Edwin H. Land and John J. McCann. 1971. Lightness and Retinex theory. *Journal of the Optical Society of America*, 61(1):1–11.

Yong Sun, Ziyi Zhao, Du Jiang, Xiliang Tong, Bo Tao, Guozhang Jiang, Jianyi Kong, Juntong Yun, Ying Liu, Xin Liu, Guojun Zhao, and Zifan Fang. 2022. Low-illumination image enhancement algorithm based on improved multi-scale Retinex and ABC algorithm optimization. *Frontiers in Bioengineering and Biotechnology*, 10:865820.

Chen Wei, Wenjing Wang, Wenhan Yang, and Jiaying Liu. 2018. Deep Retinex decomposition for low-light enhancement. In *Proceedings of the British Machine Vision Conference (BMVC)*.

Li Xu, Qiong Yan, Yang Xia, and Jiaya Jia. 2012. Structure extraction from texture via relative total variation. *ACM Transactions on Graphics*, 31(6):1–10.